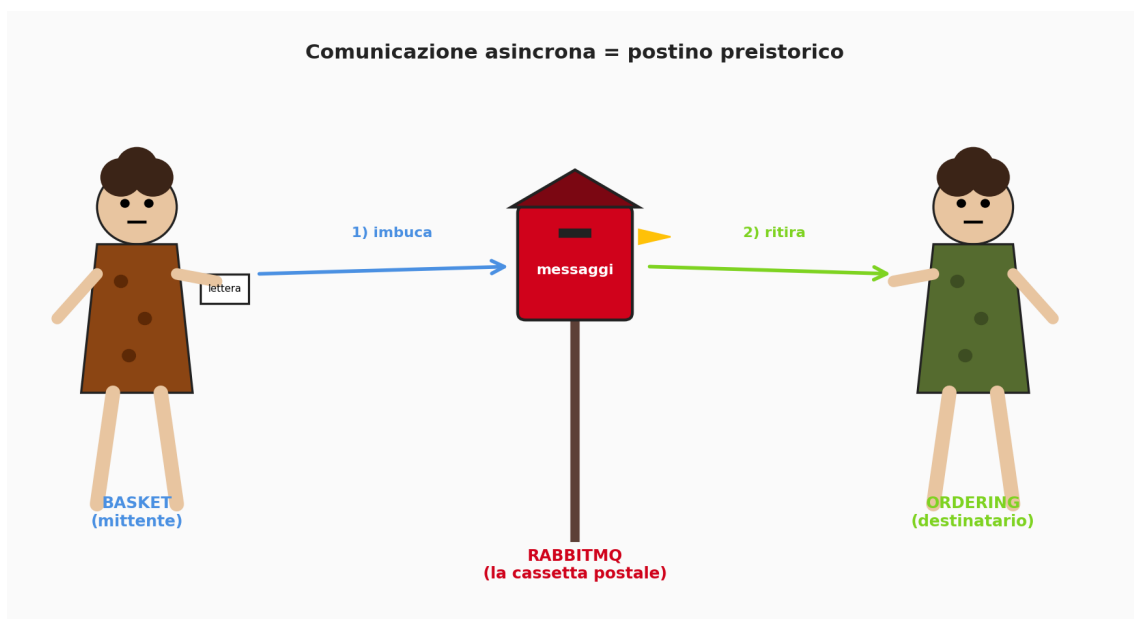


eShop

Flusso del progetto e comunicazione asincrona con il Message Broker



L'idea chiave: il Message Broker e' una cassetta postale tra due cavernicoli che non si parlano direttamente.

Documento tecnico

Architettura microservizi .NET 8 - RabbitMQ - MassTransit

Indice

1. Panoramica del progetto eShop..... 3
2. Architettura generale..... 4
3. Le tre forme di comunicazione..... 6
4. Il broker spiegato all'uomo delle caverne..... 7
5. I tre personaggi: Publisher, Broker, Consumer..... 9
6. MassTransit: chi e' e cosa fa..... 11
7. Il flusso completo del checkout (passo passo)..... 13
8. Cosa accade DENTRO RabbitMQ..... 16
9. Cosa fa MassTransit dietro le quinte (8 step)..... 17
10. Riepilogo e punti chiave da ricordare..... 18

1. Panoramica del progetto eShop

Il progetto **eShop** e' un e-commerce sviluppato in **.NET 8** seguendo lo stile architetturale a **microservizi**. Invece di una singola applicazione monolitica, ogni dominio funzionale (catalogo prodotti, carrello, ordini, sconti) vive in un servizio autonomo con il proprio database e il proprio ciclo di vita.

I quattro microservizi

Catalog API (porta 6000): gestisce i prodotti del catalogo. Persistenza su PostgreSQL tramite Marten.

Basket API (porta 6001): gestisce il carrello dell'utente. Persistenza su PostgreSQL e cache su Redis. E' anche il publisher dei nostri eventi di checkout.

Discount gRPC (porta 6002): servizio interno per il calcolo degli sconti, esposto via gRPC. Persistenza su SQLite.

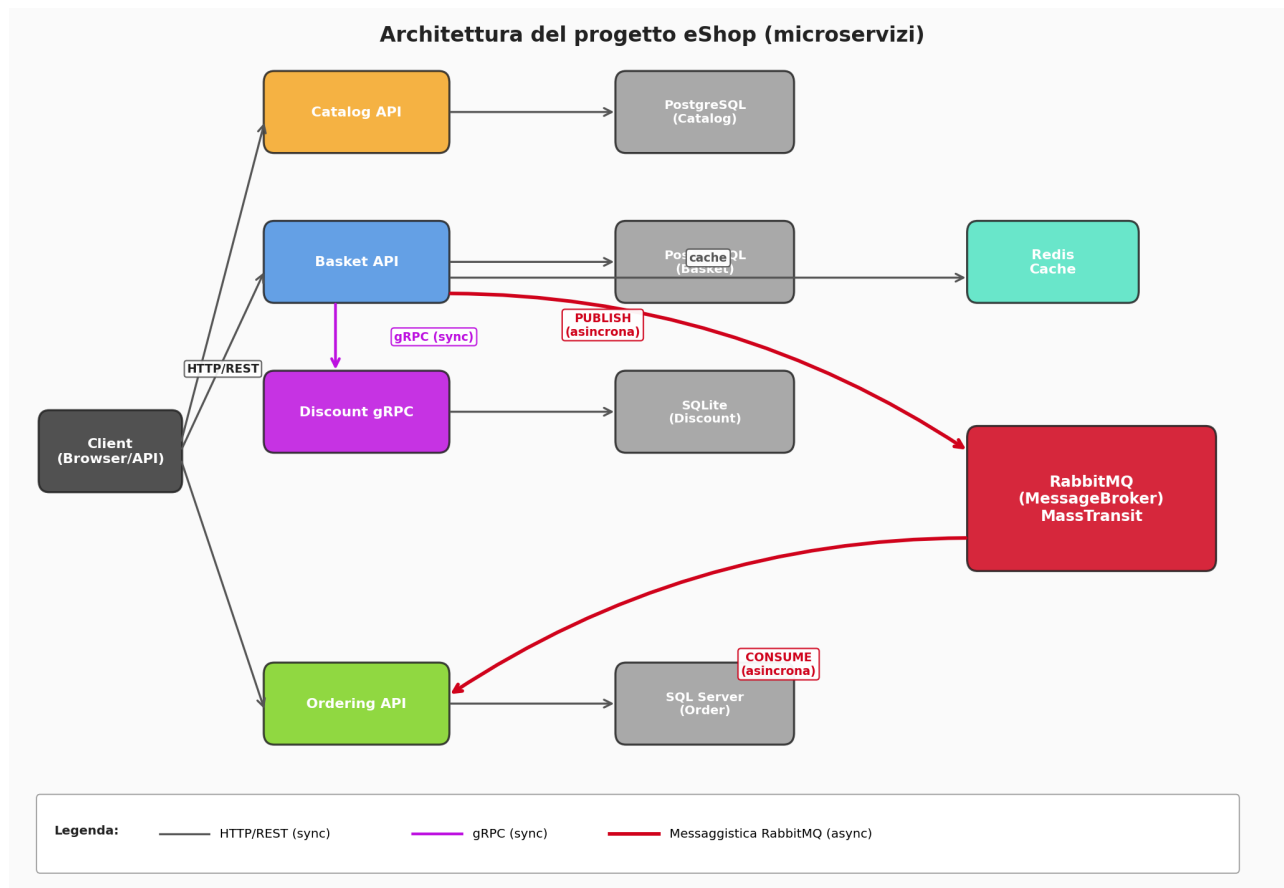
Ordering API: gestisce gli ordini. Persistenza su SQL Server, segue i pattern di **Domain-Driven Design** (aggregati, value objects, domain events) e **CQRS**. E' il consumer degli eventi di checkout.

Stack tecnologico in breve

- .NET 8, ASP.NET Core Minimal API, Carter (per gli endpoint)
- MediatR per il pattern CQRS dentro ogni servizio
- Marten + PostgreSQL (Catalog/Basket), EF Core + SQL Server (Ordering)
- Redis come distributed cache nel Basket
- gRPC per la chiamata sincrona Basket -> Discount
- **RabbitMQ** + **MassTransit** per la comunicazione asincrona Basket -> Ordering
- Docker Compose per orchestrare tutti i container

2. Architettura generale

Lo schema seguente mostra **chi parla con chi** e **come**. Le frecce nere sono chiamate HTTP/REST classiche, quelle viola sono chiamate **gRPC** (sincrone), e quelle **rosse** rappresentano i messaggi che passano dal Message Broker (asincroni).



Vista d'insieme: client, microservizi, database e RabbitMQ.

Ogni microservizio ha il **proprio database** - questo è un principio fondamentale dell'architettura a microservizi: nessuno scrive nel DB di un altro servizio. Se Ordering ha bisogno di sapere che è avvenuto un checkout, non legge dal DB del Basket: aspetta che Basket gli mandi un **messaggio**.

Perche' tante tecnologie diverse?

Ogni servizio sceglie lo strumento più adatto al proprio dominio. Il Catalog ha pochi modelli e ama PostgreSQL+Marten; Ordering ha relazioni complesse e regole di business ricche, quindi sceglie EF Core+DDD; il Basket sfrutta Redis perché i carrelli sono dati hot a vita breve. Questa eterogeneità è il prezzo (e il superpotere) dei microservizi.

3. Le tre forme di comunicazione

Nel progetto convivono tre modi diversi di far parlare due servizi. Capirne la differenza e' la chiave per capire *perche'* il Message Broker esiste.

a) HTTP/REST (sincrona)

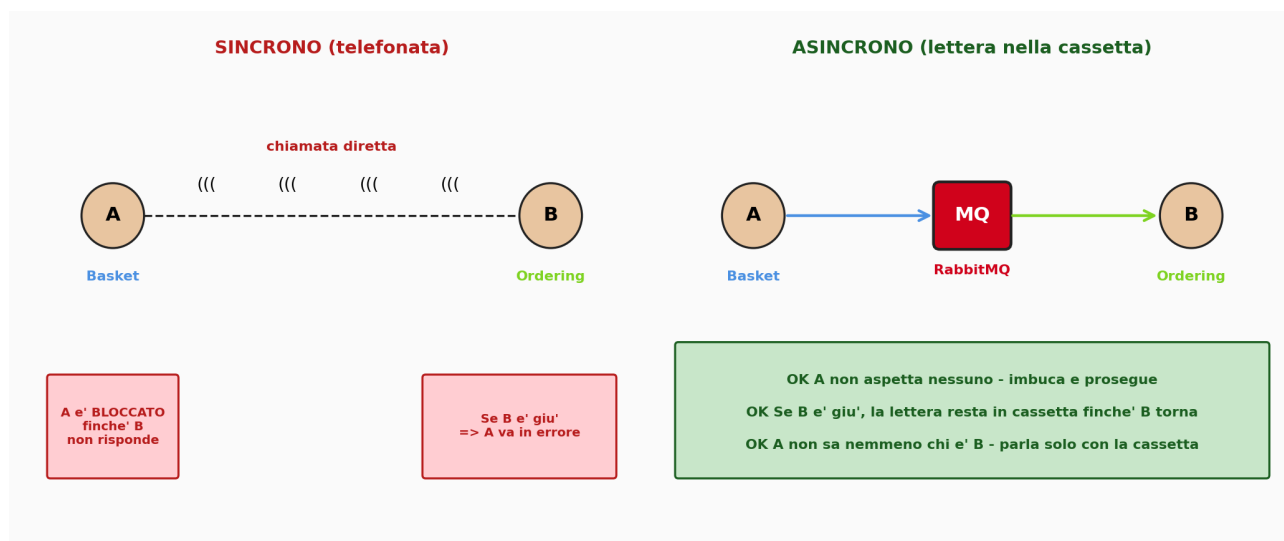
Il client (browser, Postman, ecc.) chiama via HTTP gli endpoint dei microservizi - per esempio POST /basket/checkout. E' una richiesta-risposta classica: aspetto la risposta prima di proseguire.

b) gRPC (sincrona, tra microservizi)

Quando il **Basket** ha bisogno di calcolare lo sconto su un prodotto, chiama il microservizio **Discount** via gRPC. E' anche questa una chiamata *sincrona*: Basket aspetta la risposta del servizio Discount prima di andare avanti. gRPC e' piu' veloce di HTTP/JSON perche' usa Protocol Buffers e HTTP/2, ma il principio resta lo stesso: *se Discount e' giu', Basket fallisce*.

c) Messaggistica via Broker (asincrona)

Quando il **Basket** deve dire a **Ordering** che l'utente ha completato il checkout, **non** lo chiama direttamente. Invece **pubblica un evento** su RabbitMQ. Ordering, quando puo', leggerà quell'evento e creerà l'ordine. Il Basket non sa nemmeno se Ordering esista: parla solo con la cassetta postale.



Sincrono = telefonata. Asincrono = lettera nella cassetta.

4. Il broker spiegato all'uomo delle caverne

Da qui in poi rallentiamo. Niente termini tecnici per qualche pagina: racconto la stessa cosa come se stessi parlando a un cavernicolo. Poi torniamo nei servizi.

■ L'analogia base: la cassetta postale del villaggio

Immagina due cavernicoli, **Ugo** e **Bork**, che vivono lontani. Ugo fa lance, Bork le usa per cacciare.

Ugo NON ha tempo di andare ogni giorno fino alla grotta di Bork per chiedergli se gli serve una lancia nuova. E Bork potrebbe non essere in casa, magari sta cacciando.

Allora gli abitanti del villaggio mettono in mezzo una **cassetta postale**. Quando Ugo finisce una lancia, scrive un foglietto - '*Lancia nuova pronta*' - e lo imbuca. Stop. Ugo torna a fare lance.

Bork, quando torna, passa davanti alla cassetta, prende i foglietti e fa quello che ci sta scritto.

Ugo non sa quando Bork passerà. Non gli interessa: ha consegnato il messaggio alla cassetta e ha fatto la sua parte.

■ Perché la cassetta postale è meglio del 'urlare a Bork'?

- 1) Ugo non perde tempo aspettando: imbuca e prosegue il suo lavoro.
- 2) Se Bork è malato e sta una settimana in grotta, le lettere si accumulano nella cassetta. Niente va perso.
- 3) Se in futuro arriva un terzo cavernicolo (Grug, che fa cesti) anche lui potrà iscriversi alla cassetta e leggere lo stesso foglietto. Ugo non deve cambiare nulla.
- 4) Se Ugo e Bork litigano, possono comunicare comunque: parlano alla cassetta, mai tra loro.

■ **Tradotto in termini moderni:**

- Ugo (mittente) = **Basket.API** (il servizio che pubblica)
- Bork (destinatario) = **Ordering.API** (il servizio che consuma)
- La cassetta postale = **RabbitMQ** (il Message Broker)
- Il foglietto = il **messaggio/evento** (BasketCheckoutEvent)
- Il fattore con il timbro = **MassTransit** (la libreria che gestisce le buste)

5. I tre personaggi: Publisher, Broker, Consumer

Ora che abbiamo l'immagine in testa, mettiamoci sopra i nomi tecnici. In ogni sistema basato su messaggi ci sono **sempre tre ruoli**:



I tre ruoli fondamentali nella messaggistica asincrona.

Publisher = Basket.API

È il servizio che **produce e pubblica** un evento. Nel nostro progetto il Basket pubblica un evento di tipo `BasketCheckoutEvent` ogni volta che un utente conferma il checkout.

Il publisher non sa, e non deve sapere, chi consumerà il messaggio.

Broker = RabbitMQ

È un programma a sé che gira come container Docker (vedi `docker-compose.yml`). Riceve messaggi da chiunque, li conserva in code (queue) e li consegna ai consumatori interessati. Mantiene i messaggi in memoria **e su disco** finché qualcuno non li legge (e finché non si conferma la lettura).

Consumer = Ordering.API

È il servizio che **ascolta** il broker. Nel nostro progetto il file `BasketCheckoutEventHandler` è il consumer: viene risvegliato ogni volta che arriva un messaggio dal broker e fa qualcosa con esso (creare un ordine).

i Definizione: Integration Event

Nel codice del progetto esiste una classe base `IntegrationEvent` (in `BuildingBlocks.Messaging.Events`). Si chiama *integration* perche' viaggia tra servizi diversi - integra il sistema. `BasketCheckoutEvent` eredita da questa classe.

6. MassTransit: chi e' e cosa fa

RabbitMQ da solo parla un linguaggio chiamato **AMQP**. Programmare direttamente con AMQP significa scrivere codice di basso livello: aprire connessioni, dichiarare exchange, dichiarare queue, configurare il binding, serializzare bit a bit. **MassTransit** e' una libreria .NET che fa tutto questo lavoro al posto tuo. Tu lavori con classi C#, lui parla con RabbitMQ.

Setup nel progetto: BuildingBlocks.Messaging

Il modulo condiviso BuildingBlocks.Messaging contiene un metodo di estensione AddMessageBroker che entrambi i servizi (Basket e Ordering) usano per registrare MassTransit + RabbitMQ:

```
public static IServiceCollection AddMessageBroker(
    this IServiceCollection services,
    IConfiguration configuration,
    Assembly? assembly = null)
{
    services.AddMassTransit(cfg =>
    {
        cfg.SetKebabCaseEndpointNameFormatter();
        if (assembly is not null)
            cfg.AddConsumers(assembly); // registra TUTTI gli IConsumer<T>

        cfg.UsingRabbitMq((context, configurator) =>
        {
            configurator.Host(new Uri(configuration["MessageBroker:Host"]), h =>
            {
                h.Username(configuration["MessageBroker:Username"]);
                h.Password(configuration["MessageBroker:Password"]);
            });
            configurator.ConfigureEndpoints(context);
        });
    });
    return services;
}
```

Cosa succede in queste righe:

- **SetKebabCaseEndpointNameFormatter**: il nome dell'evento in C# (BasketCheckoutEvent) viene tradotto in basket-checkout-event per i nomi delle queue.
- **AddConsumers(assembly)**: scansiona l'assembly e registra automaticamente tutte le classi che implementano IConsumer<T>. Ordering passa l'assembly di Ordering.Application perche' li' vive il BasketCheckoutEventHandler.
- **UsingRabbitMq(...)**: dice a MassTransit di usare RabbitMQ come trasporto, e configura host/credenziali leggendo da appsettings.json.
- **ConfigureEndpoints(context)**: crea automaticamente le queue per ciascun consumer trovato nel passo precedente.

Configurazione (appsettings.json)

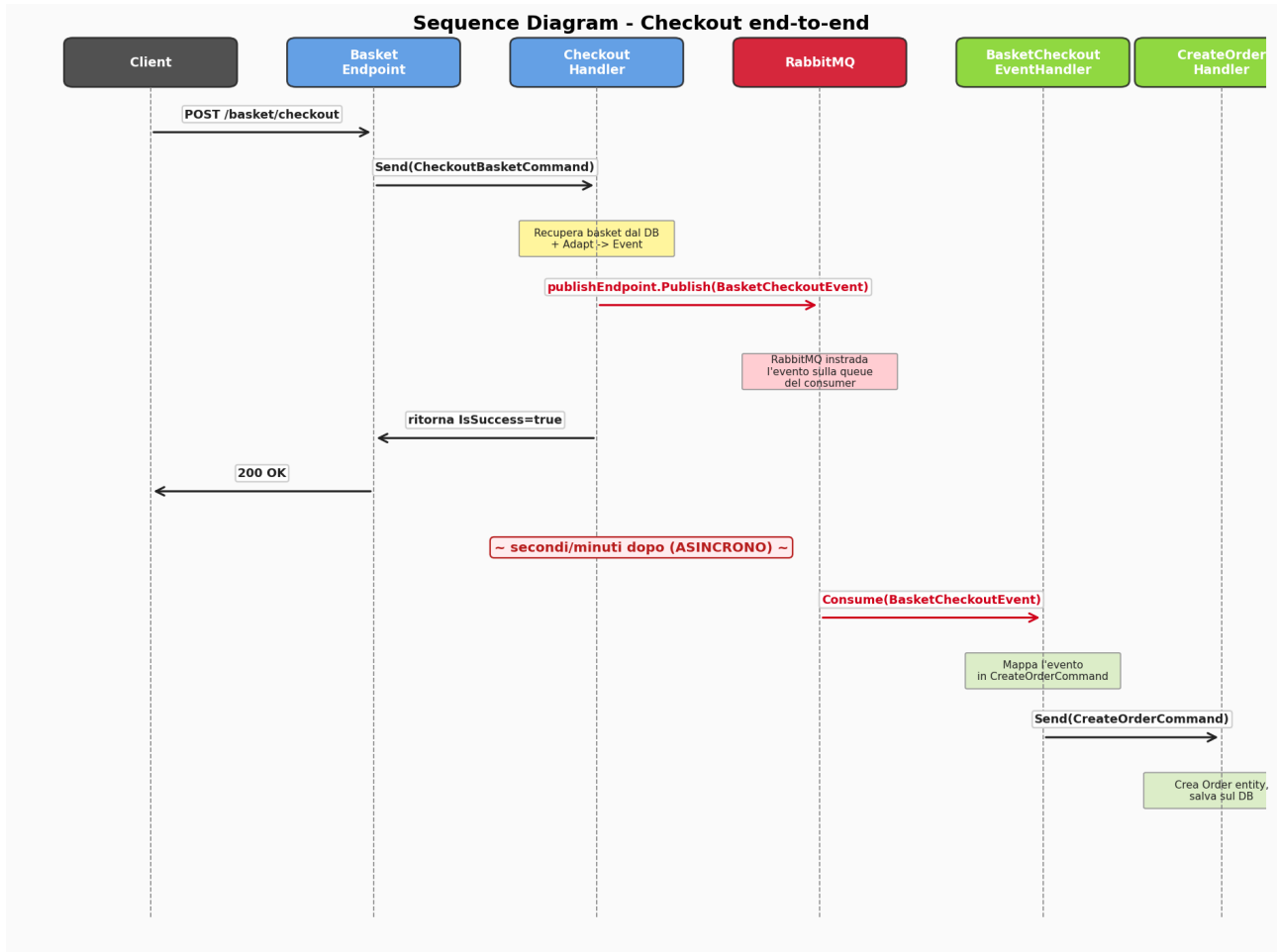
Sia Basket sia Ordering hanno questa sezione:

```
"MessageBroker": {  
  "Host": "amqp://localhost:5672",  
  "Username": "guest",  
  "Password": "guest"  
}
```

amqp://... e' l'URL del protocollo che parla con RabbitMQ. In Docker il host diventa messagebroker (il nome del container).

7. Il flusso completo del checkout (passo passo)

Mettiamo insieme tutti i pezzi. Lo scenario e': l'utente preme 'Conferma ordine' nel carrello. Vediamo cosa succede a livello di ogni servizio.



Sequence diagram: il giro del checkout dal click al record sul DB.

Step 1: Il client chiama l'endpoint Basket

Il client fa una `POST /basket/checkout` con i dati del carrello e di pagamento. L'endpoint vive in `CheckoutBasketEndpoints`:

```

app.MapPost("/basket/checkout", async (CheckoutBasketRequest request, ISender sender) =>
{
    var command = request.Adapt<CheckoutBasketCommand>();
    var result = await sender.Send(command); // -> CheckoutBasketCommandHandler
    var response = result.Adapt<CheckoutBasketResponse>();
    return Results.Ok(response);
});
  
```

Step 2: L'handler del Basket pubblica l'evento

Il vero cuore del publisher e' `CheckoutBasketCommandHandler`:

```
public class CheckoutBasketCommandHandler(
    IBasketRepository basketRepository,
    IPublishEndpoint publishEndpoint) // <-- iniettato da MassTransit
    : ICommandHandler<CheckoutBasketCommand, CheckoutBasketResult>
{
    public async Task<CheckoutBasketResult> Handle(
        CheckoutBasketCommand command, CancellationToken ct)
    {
        // 1. Recupera il basket
        var basket = await basketRepository.GetBasketAsync(
            command.BasketCheckoutDto.UserName, ct);
        if (basket is null) return new CheckoutBasketResult(false);

        // 2. Costruisce l'evento di integrazione
        var eventMessage = command.BasketCheckoutDto.Adapt<BasketCheckoutEvent>();
        eventMessage.TotalPrice = basket.TotalPrice;

        // 3. Lo PUBBLICA sul broker
        await publishEndpoint.Publish(eventMessage, ct);

        // 4. Cancella il basket dal DB
        await basketRepository.DeleteBasketAsync(
            command.BasketCheckoutDto.UserName, ct);

        return new CheckoutBasketResult(true);
    }
}
```

Tre cose da notare: **(1)** il publisher non chiama nessuna API di Ordering; **(2)** riceve un'astrazione (`IPublishEndpoint`) e non sa nulla di RabbitMQ; **(3)** finita la chiamata a `Publish`, ritorna subito al client - non aspetta che l'ordine venga creato.

Step 3: RabbitMQ riceve e instrada

MassTransit serializza l'oggetto C# in JSON e lo manda a RabbitMQ. RabbitMQ lo conserva nella coda `basket-checkout-event` (creata automaticamente). A questo punto il client ha già ricevuto un `200 OK`: il checkout, dal suo punto di vista, è concluso.

Step 4: Ordering consuma il messaggio

Il `BasketCheckoutEventHandler` in `Ordering.Application/Orders/EventHandlers/Integration` implementa `IConsumer<BasketCheckoutEvent>`. MassTransit lo invoca quando arriva un messaggio sulla queue:

```
public class BasketCheckoutEventHandler(ISender sender, ILogger<...> logger)
    : IConsumer<BasketCheckoutEvent>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        logger.LogInformation("Integration Event Handled: {Type}",
            context.Message.GetType().Name);

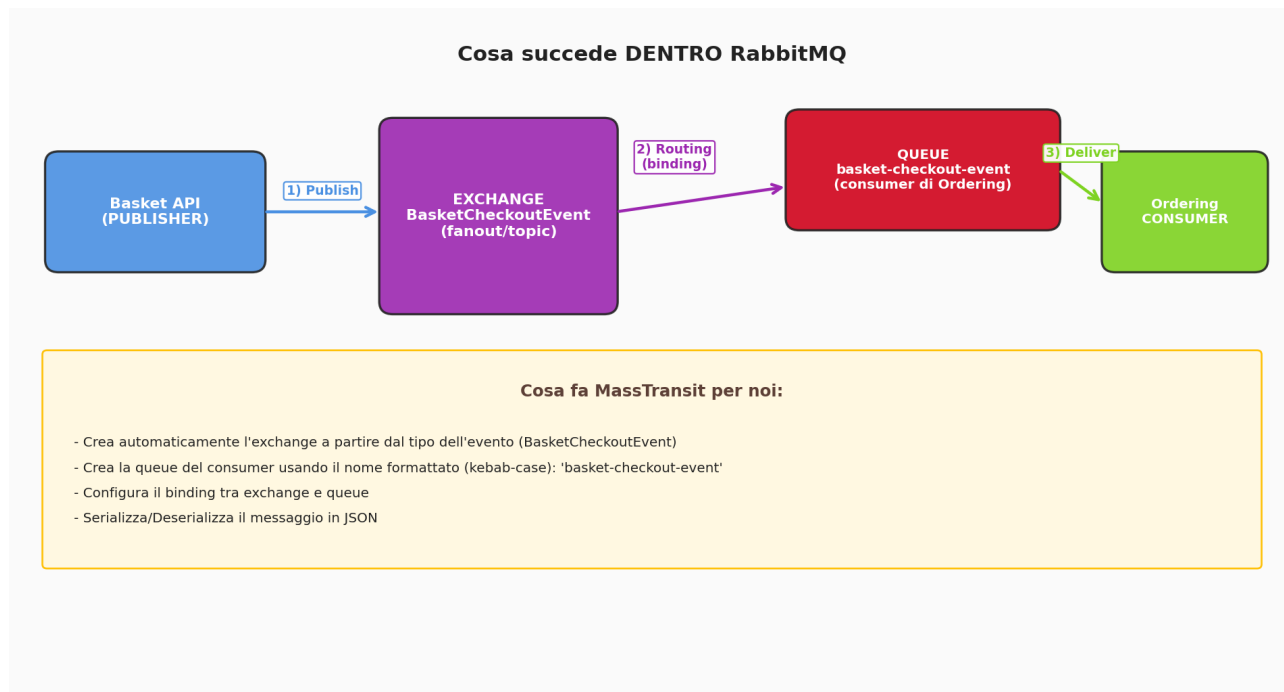
        // 1) Trasforma l'evento in un comando di dominio
        var command = MapToCreateOrderCommand(context.Message);

        // 2) Lo invia al CreateOrderHandler tramite MediatR
        await sender.Send(command);
    }
}
```

Da qui in poi siamo nel mondo Ordering: il `CreateOrderHandler` crea l'aggregato `Order` e lo persiste su SQL Server.

8. Cosa accade DENTRO RabbitMQ

Per chi vuole sapere cosa succede al messaggio dopo il Publish, RabbitMQ usa tre concetti chiave: **exchange**, **queue**, **binding**.



Exchange + Queue + Binding: il triangolo magico di RabbitMQ.

- **Exchange:** il punto in cui i messaggi *entrano*. Pensalo come lo sportello d'ingresso della cassetta. Non conserva nulla - smista soltanto.
- **Queue:** la vera 'coda' dove il messaggio aspetta finché qualcuno non lo legge. Persistita su disco se serve.
- **Binding:** la regola che dice *'i messaggi che entrano in questo exchange devono finire in questa queue'*. E' la cinghia di trasmissione.

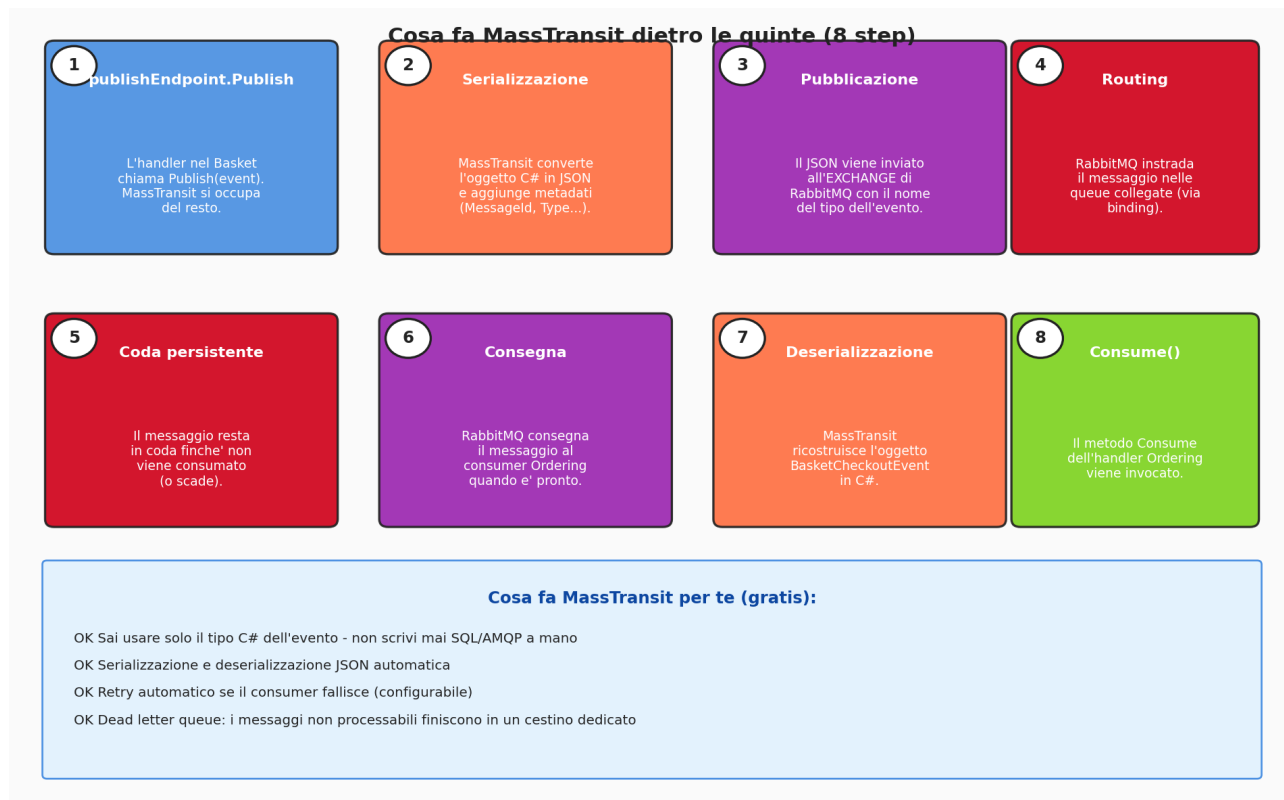
MassTransit, grazie a `SetKebabCaseEndpointNameFormatter`, crea automaticamente: un exchange chiamato come il tipo (`BasketCheckoutEvent`), una queue chiamata come il consumer (`basket-checkout-event`), e il binding che li unisce. Non scriviamo una sola riga di AMQP.

* Vuoi vedere tutto questo dal vivo?

RabbitMQ esposto sulla porta 15672 ha una **UI di management**. Apri `http://localhost:15672` (user/pass: guest/guest) e troverai tab dedicati a Exchanges, Queues, Connections. Pubblica un checkout e guarda il messaggio scorrere in tempo reale.

9. Cosa fa MassTransit dietro le quinte (8 step)

Per chiudere, ecco cosa accade dal momento in cui chiami Publish al momento in cui il tuo Consume viene invocato:



La pipeline completa: 8 step che MassTransit gestisce per te.

Step 1-4 avvengono nel processo del Basket; **step 5** vive in RabbitMQ; **step 6-8** avvengono nel processo dell'Ordering. Tu, come sviluppatore, scrivi solo lo step 1 (Publish) e lo step 8 (Consume). Tutto il resto è meccanica del framework.

+ Bonus che ottieni gratis (configurabili)

Retry: se Consume lancia un'eccezione, MassTransit ritenta automaticamente N volte prima di arrendersi.

Dead Letter Queue: i messaggi che continuano a fallire vengono spostati in una queue speciale (error) così non bloccano il flusso principale e tu puoi ispezionarli con calma.

Idempotenza: ogni messaggio ha un MessageId unico - se lo vedi due volte sai che è duplicato.

Tracing: integrazione automatica con OpenTelemetry, log strutturati e metriche.

10. Riepilogo e punti chiave da ricordare

La regola d'oro

Il publisher non parla mai direttamente al consumer. Parla solo con il broker. E il consumer non sa chi ha generato il messaggio: ascolta solo il broker. Questa indipendenza e' il motivo per cui i microservizi possono evolvere separatamente.

Le 5 cose che non devi piu' dimenticare

- **1. Asincrono = scrivere, non aspettare.** Publish ritorna subito; il lavoro vero accade dopo, nel consumer.
- **2. RabbitMQ e' fuori dalla tua app.** E' un container Docker. Tu ci parli via il protocollo AMQP - ma usando MassTransit scrivi solo classi C#.
- **3. MassTransit traduce.** Trasforma `Publish(BasketCheckoutEvent)` in comandi AMQP, e i comandi AMQP che arrivano dalla queue in chiamate al tuo `Consume(...)`.
- **4. Una queue per consumer.** Se domani aggiungi un `NotificationService` che vuole sapere quando avviene un checkout, basta creare un altro `IConsumer<BasketCheckoutEvent>`: MassTransit creera' una sua queue e riceverà una copia del messaggio.
- **5. Il broker resiste alle tempeste.** Se Ordering crolla, Basket continua a pubblicare, RabbitMQ accumula i messaggi. Quando Ordering torna su, recupera tutto cio' che ha perso.

Una mappa mentale finale

= In una frase

Il **Basket** butta una lettera nella **cassetta postale** RabbitMQ ogni volta che un cliente chiude un ordine, e **Ordering** passa di tanto in tanto a leggerla per creare l'ordine vero. **MassTransit** e' il fattore che si occupa di buste, francobolli e di consegnare a domicilio - tu scrivi solo il messaggio.

Fine del documento - Buon coding, Hassan!